Boot Controller Architecture & Data Flow

SONIC-MCU

Memory Formatting and Data Flow | SPEC-BOOTFLOW-v5.0

Introduction

The SONIC-MCU utilizes a dedicated hardware state machine to handle the bootstrapping process, circumventing the latency and complexity of a software-based Boot ROM. This Bootloader orchestrates an end-to-end Direct Memory Access (DMA) transfer.

It fetches the firmware image from a passive external SPI Flash chip, coordinates the transaction through an internal APB SPI Master IP, applies necessary endianness corrections, and autonomously writes the formatted instructions and data directly into the system's active I-SRAM and D-SRAM via the AXI-Lite crossbar.

Firmware Image and Memory Formatting

The firmware integration pipeline is a critical stage. It bridges the gap between how a standard C-compiler outputs memory arrays and how a physical SPI Flash chip expects data to be organized.

Original Word-Addressed Formats (`ebreak_imem.mem` / `ebreak_dmem.mem`)

The compiler initially outputs the compiled C-code into two distinct memory files for instructions and data. These files are formatted strictly as **32-bit, word-addressed hexadecimal files**.

Listing 1: Snippet from `ebreak_imem.mem` (Word-Addressed)

```
@00000000
00000097
@00000001
13408093
```

In this format, the "`@`" symbol denotes the 32-bit word boundary. Because RISC-V is a Little-Endian architecture, the 32-bit instruction `0x00000097` theoretically translates to four consecutive bytes in memory: `0x97`, `0x00`, `0x00`, `0x00`. However, standard SPI Flash arrays operate on a contiguous, 8-bit byte-addressed space, making this native format incompatible with a raw serial flash dump.

The Conversion Pipeline (`spi_unified_mem_instr_data.py`)

To resolve this, the build flow invokes a Python conversion script. This script executes three vital formatting transformations:

1. **Address Translation:** It maps word addresses to byte addresses (e.g., Word Address `0x00000001` logically maps to Byte Address `0x00000004`). It aggressively pads any skipped addresses in the input file with `00` to guarantee a 1:1 physical layout.
2. **Little-Endian Byte Unpacking:** It intercepts the 32-bit hex strings and decomposes them into Little-Endian byte arrays. The word `13408093` is sliced and rearranged as `93`, `80`, `40`, `13`.
3. **Unified Concatenation:** It merges the converted Instruction memory and Data memory into a single, continuous block, eliminating the need for complex multi-file handling during flash programming.

The Combined Output (`ebreak_combined.mem`)

The result of the script is the final flash image file. It lacks "`@`" address tags because it operates implicitly: line N corresponds exactly to byte address N in the SPI flash memory space.

Listing 2: Snippet from `ebreak_combined.mem` (Byte-Addressed, Little Endian)

```
97
00
00
00
93
80
40
13
```

One-Time Programmable Formatting (`s25fs512sOTP.mem`)

Conversely, internal hardware simulation models (like the `S25fs512s` simulation model) utilize specialized memory maps for their One-Time Programmable (OTP) security registers. The `s25fs512sOTP.mem` file is structured distinctly to interface with the simulator's internal loading routines:

Listing 3: Snippet from `s25fs512sOTP.mem`

```
@000
00
01
02
@010
9F
```

While this file is also byte-oriented, the simulator's `$readmemh` parser expects explicit byte-address anchors (e.g., `@000`, `@010`) to map these isolated security bytes directly into internal status registers rather than the main sequential flash array.

Important

Necessity of the OTP File: The OTP memory contains critical configuration bytes, non-volatile lock bits, and device identification parameters. In physical hardware, these are programmed at the factory or during device provisioning. In the verification environment, the `s25fs512s` simulation model strictly requires the `s25fs512sOTP.mem` file to preload these internal states. If this file is omitted or incorrectly formatted, the flash model will initialize in an undefined or locked state, causing it to reject incoming SPI read commands or assert false protection faults, leading to simulated boot failures.

System Reset and Boot Orchestration

In the SONIC-MCU architecture, the Bootloader does not act merely as a data pump; it serves as the Master Reset and Boot Controller for the entire System-on-Chip (SoC).

Waking up the RISC-V core and the Bootloader simultaneously without sequencing would induce immediate contention on the AXI-Lite crossbar. To prevent deadlocks and mitigate dynamic power glitches from

uninitialized logic matrices, the Bootloader hardware actively halts the core and non-essential peripherals during memory population.

Dual Reset Topology and Memory Retention

The system provides two distinct external reset paths that dictate the Bootloader's execution flow:

- **Cold Boot (`hard_rst_ni`):** Initiated during a full hardware power-up. The Bootloader executes a complete initialization: sanitizing the SRAM, waiting for external flash power-up, configuring the SPI Master via APB, and orchestrating the DMA fetch loop to populate memory.
- **Warm Boot (`soft_rst_ni`):** Initiated for a rapid system restart. This path protects existing SRAM integrity by clamping the SRAM write-enable signals to zero. The FSM skips the SPI flash fetch entirely, allowing the CPU to instantly re-execute previously loaded firmware.

Boot Controller State Machine

The flowchart below illustrates how the Boot FSM branches dynamically based on the reset type before converging on the clean-slate hardware release.

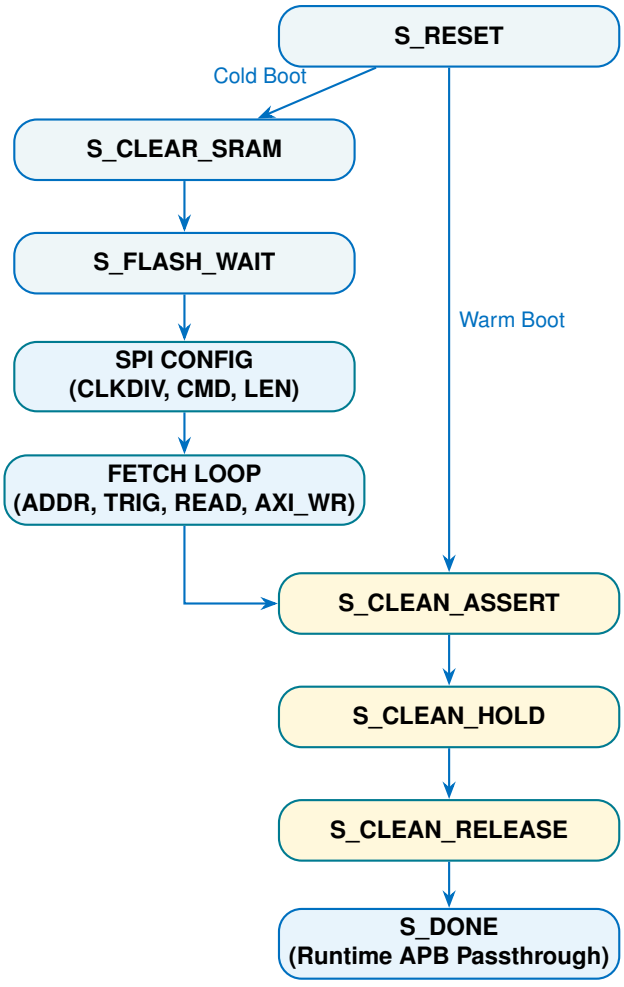


Figure 1: Overall Boot and Reset Sequence FSM

Post-Load Clean-Slate Sequence

A critical requirement of the boot architecture is ensuring that no leftover bootloader configuration (such as the SPI Clock Divider or active Chip Select states) leaks into the CPU's runtime execution environment.

Before entering the terminal S_DONE state in either a cold or warm boot, the FSM must traverse the clean-slate sequence (S_CLEAN_ASSERT → S_CLEAN_HOLD → S_CLEAN_RELEASE). This sequence simultaneously pulses core_rst_ni, periph_rst_ni, and spi_rst_ni LOW for a parameterized number of clock cycles. This guarantees a deterministic, blank state for the SoC.

```
unique case (state_d)
  S_CLEAR_SRAM, S_FLASH_WAIT, S_INIT_CLKDIV: begin
    spi_rst_q <= 1'b1;
  end
  S_CLEAN_ASSERT, S_CLEAN_HOLD: begin
    core_rst_q   <= 1'b0;
    periph_rst_q <= 1'b0;
    spi_rst_q    <= 1'b0;
  end
  S_CLEAN_RELEASE, S_DONE: begin
    core_rst_q   <= 1'b1;
    periph_rst_q <= 1'b1;
    spi_rst_q    <= 1'b1;
  end
  default: ;
endcase
```

Dynamic APB Multiplexing

The Bootloader physically sits between the system's APB Bridge (Slot 4) and the apb_spi_master IP.

- **During Boot (boot_active = 1):** The FSM directly drives the APB master port to configure the SPI registers and execute the hardware DMA. Any incoming APB traffic from the slave port is safely met with an idle response.
- **At Runtime (S_DONE):** The FSM relinquishes bus control, creating a transparent passthrough. The CPU can now communicate directly with the SPI Master over the AXI-Lite crossbar and APB bridge as if the Bootloader hardware were not there.

Decoupled Hardware Synchronization

The boot flow relies on two distinct Finite State Machines executing simultaneously to achieve hardware-level synchronization without the need for software polling loops.

Parallel FSM Execution

- **SPI Master IP:** Once triggered by the Boot Controller, the SPI IP transitions out of its idle state. It autonomously shifts data bit-by-bit and subsequently pushes formatted 32-bit words directly into its RX FIFO.
- **Boot Controller:** Following the trigger, the Boot Controller attempts to read the RX FIFO via the APB bus. If the FIFO is empty, the APB bus gracefully stalls by pulling PREADY to '0'. The Boot Controller passively sleeps in this state. The exact cycle the SPI Master IP pushes a completed word into the FIFO, PREADY asserts to '1', unblocking the FSM to capture the data and execute the AXI-Lite write.

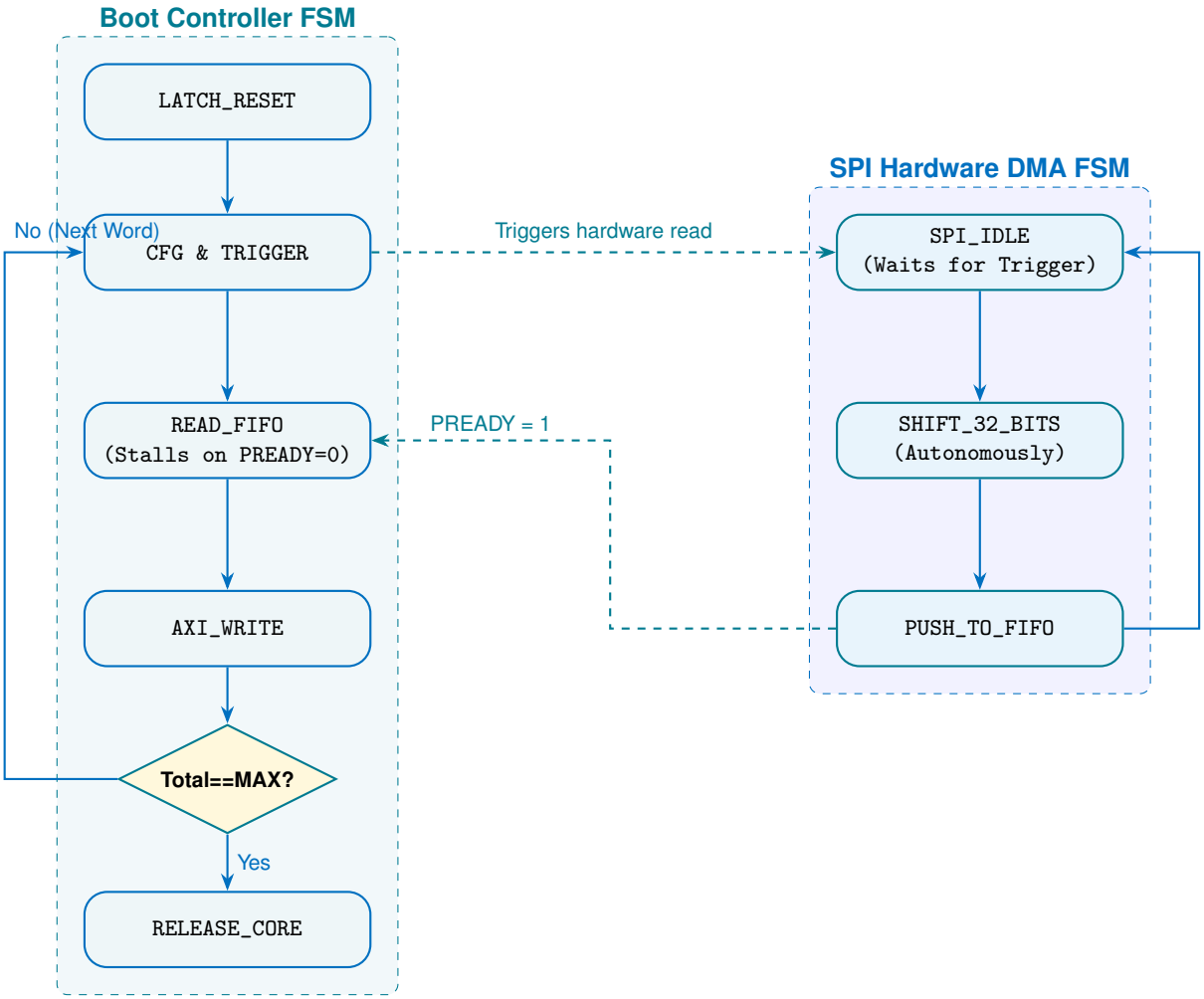


Figure 2: Parallel FSMs: Decoupled Hardware Synchronization

Boot Execution Flows and Resets

The Bootloader sits between the APB bridge and the SPI master, dynamically selecting between two initialization paths based on the external reset inputs:

- **Cold Boot:** Initiated by a `hard_rst_ni` transition from LOW to HIGH. The flow advances through `S_RESET` to `S_CLEAR_SRAM`, handles the SPI configuration, reads the firmware word-by-word, and executes a final clean-slate reset block (`S_CLEAN_ASSERT` to `S_CLEAN_RELEASE`) before entering `S_DONE`. Crucially, the internal RTL explicitly holds the SPI master in reset upon entry by keeping `spi_rst_q` LOW, releasing it to HIGH only when transitioning past `S_RESET`.
- **Warm Boot:** Initiated by a `soft_rst_ni` transition from LOW to HIGH. The system actively skips the SPI fetch phase to preserve current SRAM contents. The flow executes `S_RESET`, jumps directly to the clean-slate sequence, and terminates at `S_DONE`.

End-to-End Data Flow Pipeline

The journey of a single 32-bit firmware word follows this physical and logical route from the non-volatile storage to the active system memory:

1. **Stage 1: Flash to SPI Master (Physical Layer):** The external Flash chip shifts data out serially on its `SO` pin. This enters the SoC via the SPI Master's `spi_sdi1_w` pin. The Master captures 32 bits and pushes them to its APB `RX_FIFO`.

2. **Stage 2: SPI Master to Bootloader (APB Protocol):** The Bootloader, stalled in the `S_READ_FIFO` phase, sees the `PREADY` signal go HIGH. It captures the data from the FIFO across the APB bus.
3. **Stage 3: Endianness Correction & AXI-Lite Target:** Because the data was packed byte-by-byte into the flash (Byte 0 first), it must be reversed. The Bootloader swaps the byte lanes (`[7:0]` to `[31:24]`, etc.), reconstructing the native Little-Endian format. It is then placed on the AXI-Lite `WDATA` channel, alongside the target SRAM address on the `AWADDR` channel, routing the packet directly into the destination memory.

Hardware Synchronization Timing (Waveform)

The waveform below details the timing relationship across the independent buses. It proves how the Boot Controller's APB read transaction is safely suspended, absorbing the slow SPI transaction time natively, and immediately responding the cycle the data arrives.

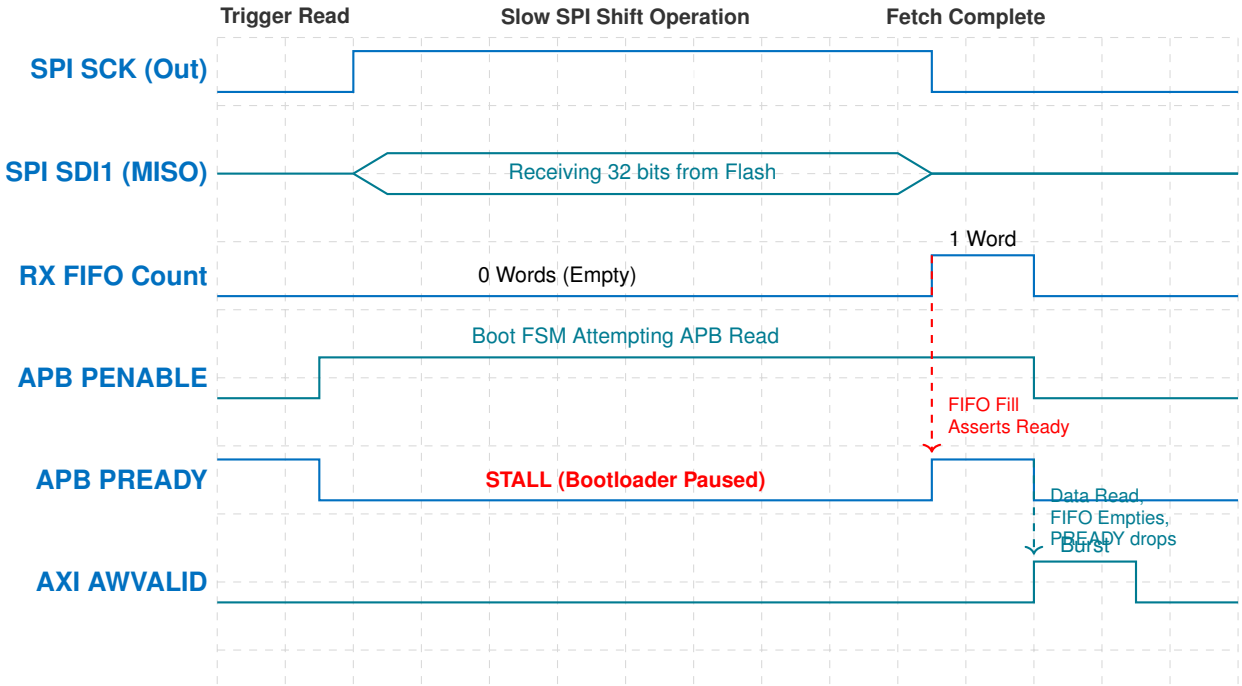


Figure 3: Cycle-Accurate Hardware Timing Sequence

Note

Crucially, when the Bootloader successfully reads the APB bus at cycle 12, it drains the `RX_FIFO`. As the FIFO Count immediately falls back to 0, the SPI Master's internal bus logic concurrently drives `PREADY` back to 0, safely clearing the bus for the next transaction and preventing accidental double-reads.

RTL Implementation & Code Flow

The bootloader is implemented in SystemVerilog utilizing a standard multi-process Finite State Machine architecture. The logic is distinctly separated into sequential state updates, combinational next-state decoding, and active bus driving blocks.

State Register and Reset Handling

The sequential block is clocked on the positive edge of the system clock and handles asynchronous resets. Crucially, it evaluates both `hard_rst_ni` and `soft_rst_ni` to determine whether the system is undergoing a cold or warm boot.

Listing 4: FSM State Register Update

```
always_ff @(posedge clk_i or negedge hard_rst_ni or negedge soft_rst_ni) begin
  if (!hard_rst_ni) begin
    state_q <= S_RESET;      // Cold boot: start from scratch
    // Reset internal counters...
  end else if (!soft_rst_ni) begin
    state_q <= S_DONE;       // Warm boot: bypass SPI, jump to DONE
    // Preserve counters...
  end else begin
    state_q <= state_d;      // Normal operation: transition to next state
    // Update counters...
  end
end
```

Next State Logic (Combinational FSM)

The transition logic dictates how the bootloader advances through its pipeline. It continuously monitors the handshakes from both the APB and AXI domains (e.g., `pready`, `aw_ready`, `w_ready`) to ensure it does not proceed until transactions are properly acknowledged by the external bus structures.

Listing 5: Combinational Next-State Evaluation

```
always_comb begin
  state_d = state_q; // Default: hold state

  case (state_q)
    S_CFG_SPIADR: begin
      // Wait until the APB write command is accepted
      if (apb_mst_resp_i.pready) state_d = S_TRIGGER;
    end
    S_TRIGGER: begin
      if (apb_mst_resp_i.pready) state_d = S_READ_FIFO;
    end
    S_READ_FIFO: begin
      // Stall here until the SPI IP pushes data and asserts PREADY
      if (apb_mst_resp_i.pready) state_d = S_WRITE_SRAM;
    end
    S_WRITE_SRAM: begin
      // Wait for AXI crossbar to accept the write burst
      if (axi_aw_ready && axi_w_ready) state_d = S_CHECK_DONE;
    end
    // ... (other states omitted for brevity)
  endcase
end
```

Bus Interface and Endianness Routing

The output logic actively drives the APB and AXI buses based on the current state. When reading from the SPI `RX_FIFO` (`S_READ_FIFO`), the hardware captures the raw serial payload. Before writing this data to the SRAM (`S_WRITE_SRAM`), the RTL manually restructures the bytes to bridge the Endianness gap between the SPI Flash format and the RISC-V CPU execution format.

Listing 6: Endianness Correction and AXI Routing

```
// Capture data from APB when PREADY asserts
if (state_q == S_READ_FIFO && apb_mst_resp_i.pready) begin
  // Perform Little-Endian byte-swap natively in hardware
  rxfifo_data_q <= { apb_mst_resp_i.prdata[7:0],
                     apb_mst_resp_i.prdata[15:8],
                     apb_mst_resp_i.prdata[23:16],
                     apb_mst_resp_i.prdata[31:24] };
```

```
end

// Drive AXI-Lite Write Data Channel
assign axi_req_o.w.data = rxfifo_data_q;
assign axi_req_o.w_valid = (state_q == S_WRITE_SRAM);
```

Boot Performance & Hardware Metrics

During physical RTL verification (`xrun.log` dated Jul 24, 2026), cycle-accurate performance measurements were captured to validate the hardware's temporal efficiency.

The testbench operates on a system clock period of $T_{clk} = 10$ ns (100 MHz base frequency). The APB SPI Master Clock Divider (`SPI_CLK_DIV`) is parameterized to a constant value of **4**, generating a 12.5 MHz physical SPI Serial Clock. The firmware footprint utilized for the benchmark was exactly **6,156 bytes** (1,539 words).

Table 1: Cycle-Accurate Boot Latency Breakdown

Boot Phase	Clock Cycles	Absolute Time
Configuration Phase	35,135	351.34 μs
SRAM Feeding Phase	989,583	9.89 ms
Total Boot Latency	**1,024,718**	**10.24 ms**

- **Configuration Phase:** Encompasses the `S_CLEAR_SRAM` pass (zeroing out the entire 2,560 words of target memory bounds to prevent X-states) and the `S_FLASH_WAIT` power-up stall mandated by the vendor's physical flash simulation model (30,000 cycles).
- **SRAM Feeding Phase:** The active DMA transfer window fetching all 1,539 words sequentially across the SPI bus and dispatching them via AXI-Lite.
- **Absolute Determinism:** The decoupled hardware FSM architecture guarantees absolute temporal determinism. Repeating the cold boot sequence continuously yields an identical footprint of exactly 1,024,718 cycles without deviation.